算法设计与分析

计算机学院

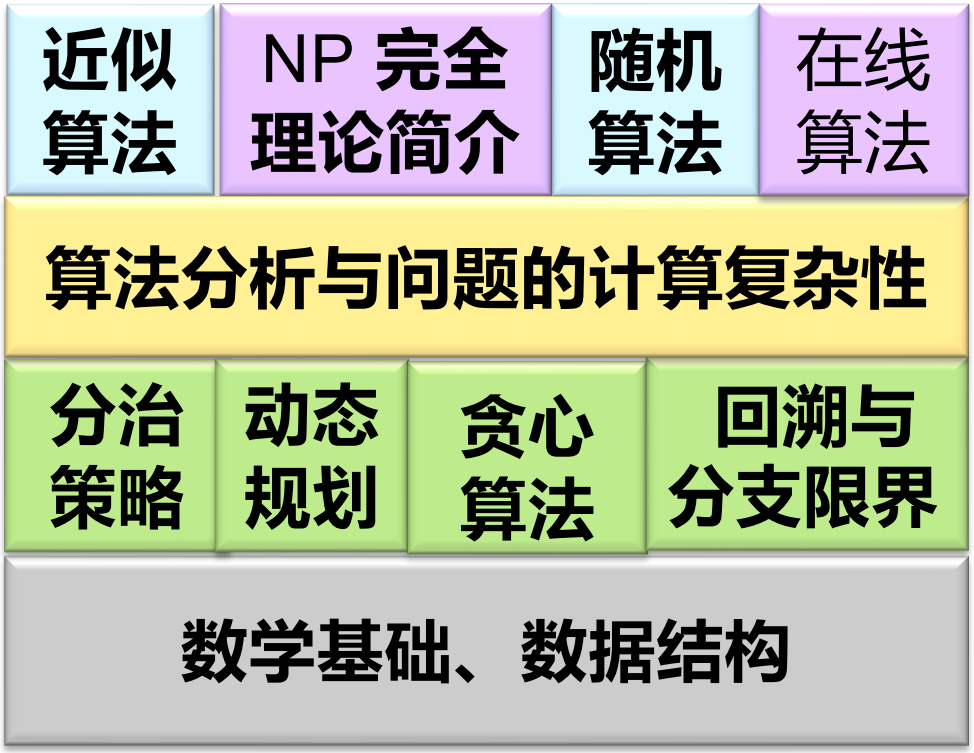
课程信息

- 上课时间和地点：
  - 大班课：每周一**3-4**节，单周三**5-6**节，二教**209**

- 授课老师：汪小林
  单位：计算机学院 网络与高能效计算研究所
  研究方向：操作系统、系统虚拟化、软硬件协同设计
  电子邮件：**wxl@pku.edu.cn**

- 助教：

课程内容



问题处理策略

计算复杂性理论

算法分析方法

算法设计技术

基础知识

教材

书名：

《算法设计与分析》

作者：

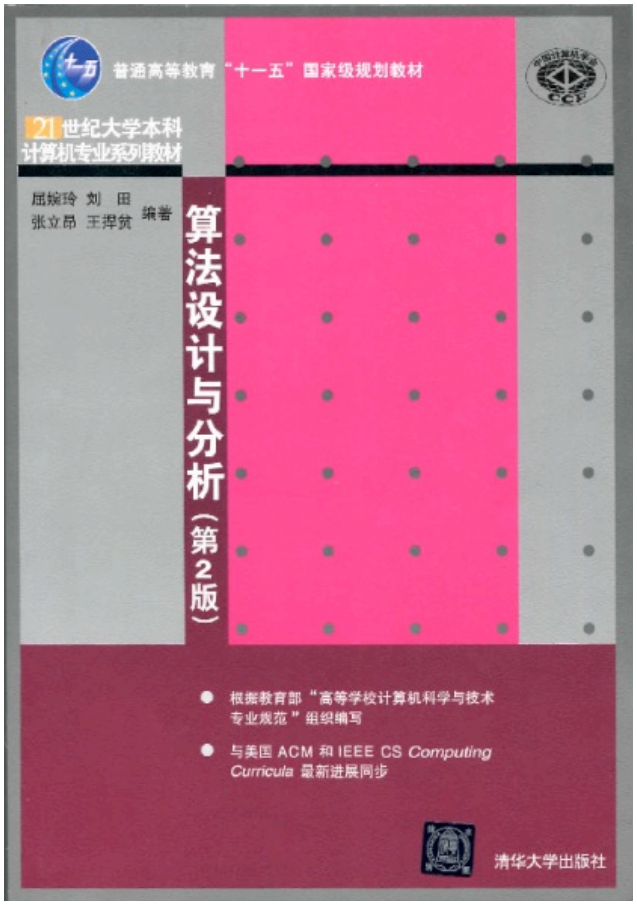
屈婉玲, 刘田, 张立昂, 王捍贫

出版社：

清华大学出版社

出版时间：

2016年第2版



参考书

1. Jon Kleinberg, Eva Tardos, Algorithm Design, Addison-Wesley, 清华大学出版社影印版，2006.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L.Rivest, Introduction to Algorithms(3rd edtion), The MIT Press 2009.
3. 张立昂，可计算性与计算复杂性导引（第3版），北京大学出版社，2011.
4. 堵丁柱，葛可一，王洁，计算复杂性导论，高教出版社2002.
5. Sanjeev Arora, Boaz Barak, Computational Complexity: A Modern Approach, Cambridge University Press，2009.
6. 屈婉玲, 刘田, 张立昂, 王捍贫, 算法设计与分析习题解答与学习指导，清华大学出版社，2016年第2版

学习安排

- 授课形式
  - 大班上课：布置书面作业

- 成绩评定：
  - 期中考试(30%)
  - 期末考试（50%）
  - 作业（20%）

  - 缓考重修的同学，请联系助教说明以前上课的年份和大班老师，以便查找以前的成绩

引言：理论上的可计算与现实上的可计算

- 算法研究的重要性
- 理论上的可计算
  ——可计算性理论
- 现实上的可计算
  ——计算复杂性理论

几个例子

例1：求解调度问题

任务集 $S=\{1, 2, \ldots, n\}$，

第 j 项加工时间：$t_j, \in Z^+, j=1, 2, \ldots, n$

一个可行调度方案：$1, 2, \ldots, n$ 的排列 $i_1, i_2, \ldots, i_n$

求：总等待时间最少的调度方案，即求

$$S\text{的排列}i_1, i_2, \ldots, i_n\text{使得}\quad \min\{\sum_{k=1}^{n}(n-k+1)t_{i_k}\}$$

求解方法

贪心策略：加工时间短的先做

如何描述这个方法？这个方法是否对所有的实例都能得到最优解？如何证明？这个方法的效率如何？

例2：投资问题

问题：

m元钱，投资给n个项目，效益函数 $f_i(x)$，表示第 i 个项目投入x元钱的效益，i=1,2,…,n. 如何分配每个项目的钱数使得总效益最大？

令x_i是第 i 个项目的钱数

$$\max \sum_{i=1}^{n} f_i(x_i),$$

$$\sum_{i=1}^{n} x_i = m, \quad x_i \in \mathrm{N}$$

采用什么算法？效率怎样？

蛮力算法的代价

Stirling公式：$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1+\Theta\left(\frac{1}{n}\right)\right)$

非负整数解 $<x_1, x_2, \ldots, x_n>$ 的个数估计：

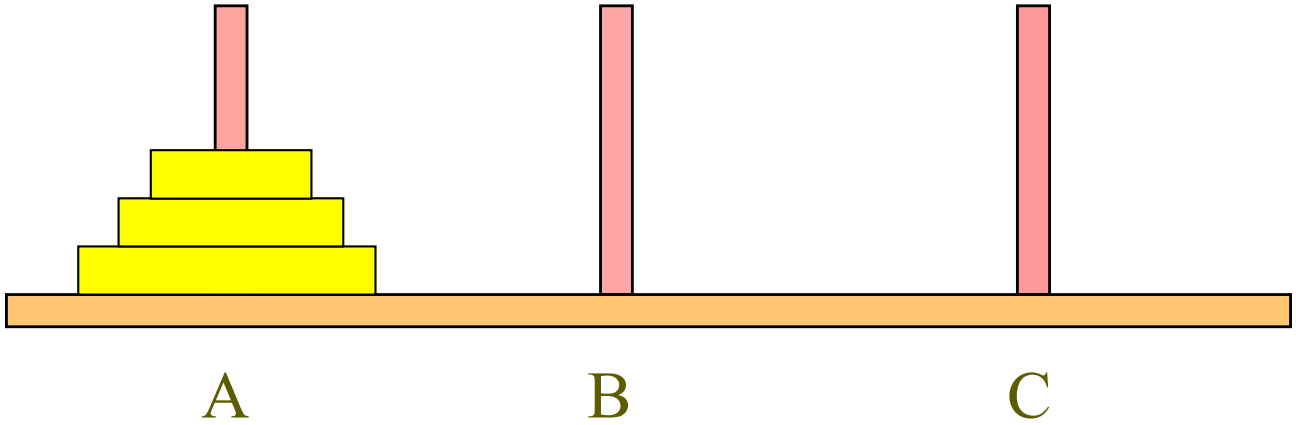
$$C_{m+n-1}^{m} = \frac{(m+n-1)!}{m!(n-1)!} = \Omega((1+\varepsilon)^{m+n-1})$$

蛮力算法——穷举法代价太大

能否利用解之间的依赖关系找到更好的算法？

结论：需要算法设计技术

例3 Hanoi塔问题



$T(n) = 2\,T(n-1) + 1$，$T(1) = 1$，解得 $T(n) = 2^n - 1$

1秒移1个，64个盘子要多少时间？（5000亿年），千万亿次/秒，4个多小时.

思考：是否存在更好的解法？

Reve难题：Hanoi塔变种，柱数增加，允许盘子相等.

其他变种：奇偶盘号分别放置

例4 排序算法的评价

已有的排序算法：考察元素比较次数

插入排序、冒泡排序：最坏和平均状况下都为$O(n^2)$

快速排序：最坏状况为$O(n^2)$，平均状况下为$O(n\log n)$

堆排序、二分归并排序：最坏和平均状况下都为$O(n\log n)$

…

问题

哪个排序算法效率最高？

是否可以找到更好的算法？排序问题的计算难度如何估计？

Algorithm + Data Structure = Programming

- 好的算法
  - 提高求解问题的效率
  - 节省存储空间
- 需要解决的问题
  - 问题→寻找求解算法　　算法设计技术
  - 算法→算法的评价　　算法分析技术
  - 算法类→问题复杂度的评价　　问题复杂性分析
  - 问题类→能够求解的边界　　计算复杂性理论

算法研究的重要性

- 算法设计与分析技术在计算机科学与技术领域有着重要的应用背景
- 算法设计分析与计算复杂性理论的研究是计算机科学技术的核心研究领域
  - **1966-2005**期间，**Turing**奖获奖**50**人，其中**10**人以算法设计，**7**人以计算理论、自动机和复杂性研究领域的杰出贡献获奖
  - 计算复杂性理论的核心课题 “**P=NP?**” 是本世纪**7**个最重要的数学问题之一
- 通过算法设计与分析课程的训练对提高学生的素质和分析问题解决问题的能力，对培养计算思维有着重要的作用

理论上的可计算：可计算性理论

- 研究目标

  确定什么问题是可计算的，即存在求解算法

- 合理的计算模型

  已有的：递归函数、Turing机、λ演算、Post系统等

  条件：计算一个函数只要有限条指令

  每条指令可以由模型中的有限个计算步骤完成

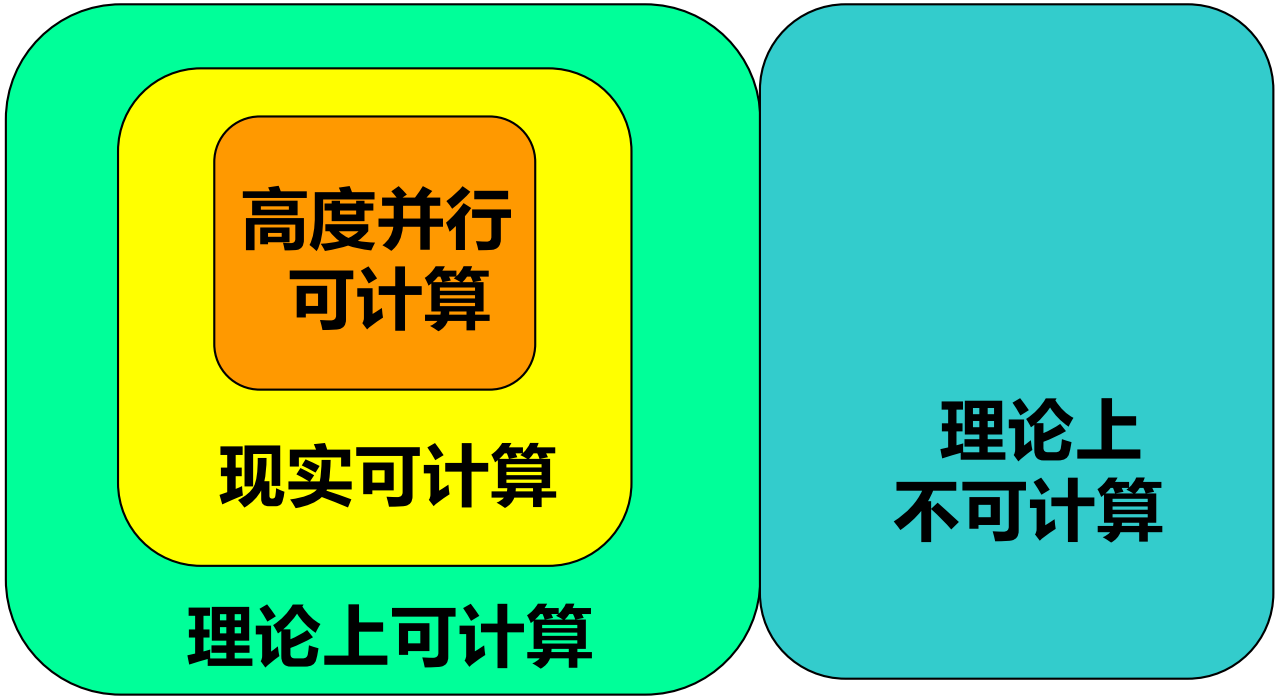
  指令执行的过程是确定的

- 核心论题：**Church-Turing论题**

  如果一个函数在某个合理的计算模型上可计算，那么它在Turing机上也是可计算的

- 可计算性是不依赖于计算模型的客观性质

理论上与现实上的可计算性



算法至少具有指数时间：理论上可计算——难解的

多项式时间的算法：现实上可计算——多项式时间可解的

对数多项式时间的算法：高度并行可解的

计算复杂性理论

内容

- 算法复杂度——算法所使用的时间、空间的估计
- 问题复杂度——估计问题的难度

术语和概念

- 问题
- 算法
- 算法的时间复杂度
- 函数的阶
- 多项式时间的算法与指数时间的算法
- 问题的复杂度分析

问题

问题：

需要回答的一般性提问，通常含有若干参数

问题描述所包含的内容：

对问题参数的一般性描述

解满足的条件

问题的实例：

对问题的参数的一组赋值

一个问题是由它的全体实例构成的集合

算法

非形式定义

有限条指令的序列

确定了解决某个问题的运算或操作

输入个数大于等于0

输出个数大于0

形式定义

对所有的有效输入停机的Turing机

算法 *A* 解问题 *P*

把问题*P*的任何实例作为算法*A*的输入，*A*能够在有限步停机，并输出该实例的正确的解

算法的描述：伪码

- 保持程序的主要结构
  - 类 C, Pascal
  - 赋值语句：←
  - 分支语句：if …then …[else…]
  - 循环语句：while, for，repeat ..until
  - 转向语句：goto
  - 调用
  - 注释：//…
- 允许使用自然语言
- 常忽略数据结构、模块、异常处理等细节
- 常忽略变量说明

伪码的例子：插入排序

算法 **INSERTION-SORT(*A*)**

```
for j ← 2 to length[A]
    do key ← A[j]      // 将A[j]插入排好序的序列 A[1..j–1]
        i ← j–1
        while i > 0 and A[i] > key
            do  A[i+1] ← A[i]
                i ← i –1
        A[i+1] ← key
```

算法的时间复杂度

- 最坏情况下的时间复杂度

  算法求解输入规模为n的实例所需要的最长时间$W(n)$

- 平均情况下的时间复杂度

  算法求解输入规模为n的实例所需要的平均时间$A(n)$

$$A(n) = \sum_{I \in S} t_I p_I$$

S:实例集，t_I: 实例I的基本运算次数，p_I: I的概率

- 复杂度表示

  针对问题选择基本运算

  将基本运算次数表示为输入规模的函数

实例

搜索问题

输入：非降顺序排列的数组L，元素数为n； 数x

输出：j. 若x在L中，j 是 x 首次出现的序标；否则 $j=0$

算法 顺序搜索

假设 x 在 L 中的概率为 p

x 在 L 中不同位置是等概分布的，则

$$W(n)=n$$

$$A(n)=\sum_{i=1}^{n} i\frac{p}{n}+(1-p)n=\frac{p(n+1)}{2}+(1-p)n$$

函数渐近的界

设 f 和 g 是定义域为自然数集 N上的函数

(1) $f(n)=O(g(n))$

若存在正数c和n_0使得对一切$n\geq n_0$有$0\leq f(n)\leq cg(n)$

(2) $f(n)=\Omega(g(n))$

若存在正数c和n_0使得对一切$n\geq n_0$有$0\leq cg(n)\leq f(n)$

(3) $f(n)=o(g(n))$

对所有正数$c>0$存在n_0使得对一切$n\geq n_0$有$0\leq f(n)<cg(n)$

(4) $f(n)=\omega(g(n))$.

对所有正数$c>0$存在n_0使得对一切$n\geq n_0$有$0\leq cg(n)<f(n)$

(5) $f(n)=\Theta(g(n)) \Leftrightarrow f(n)=O(g(n))$ 且 $f(n)=\Omega(g(n))$

(6) $O(1)$表示常数函数

函数渐近的界的基本性质

定理1.1 设 f 和 g 是定义域为自然数集 N上的函数.

(1) 如果 $\lim\limits_{n\to\infty}\dfrac{f(n)}{g(n)}$ 存在，并且等于某个常数$c>0$，那么

$f(n)=\Theta(g(n))$.

(2) 如果$\lim\limits_{n\to\infty}\dfrac{f(n)}{g(n)}=0$，那么

$f(n)=o(g(n))$.

(3) 如果 $\lim\limits_{n\to\infty}\dfrac{f(n)}{g(n)}=+\infty$，那么

$f(n)=\omega(g(n))$.

证明定理1.1(1)

(1) 根据极限定义，对于给定的正数$\varepsilon=c/2$, 存在某个n_0，只要$n\geq n_0$，就有

$$\left|\frac{f(n)}{g(n)}-c\right|<\varepsilon \Rightarrow c-\varepsilon<\frac{f(n)}{g(n)}<c+\varepsilon$$

$$\Rightarrow \frac{c}{2}<\frac{f(n)}{g(n)}<\frac{3c}{2}<2c$$

对所有的$n\geq n_0$， $f(n)\leq 2cg(n)$. 从而推出 $f(n)=O(g(n))$

对所有的$n\geq n_0$， $f(n)\geq (c/2)g(n)$，从而推出 $f(n)=\Omega(g(n))$, 于是 $f(n)=\Theta(g(n))$

函数渐近的界的基本性质

定理1.2 设 f, g, h是定义域为自然数集合的函数，

(1) 如果 $f=O(g)$且 $g=O(h)$，那么 $f=O(h)$.

(2) 如果 $f=\Omega(g)$且 $g=\Omega(h)$，那么 $f=\Omega(h)$.

(3) 如果 $f=\Theta(g)$和 $g=\Theta(h)$，那么 $f=\Theta(h)$.

定理1.3 假设 f 和g是定义域为自然数集合的函数，若对某个其它的函数 h, 我们有$f=O(h)$和$g=O(h)$，那么

$$f+g=O(h).$$

推论 假设 f 和g是定义域为自然数集合的函数，且满足 $g=O(f)$，那么 $f+g=\Theta(f)$.

基本函数类

阶的高低

至少指数级：$2^n, 3^n, n!, \ldots$

多项式级：$n, n^2, n\log n, n^{1/2}, \ldots$

对数多项式级：$\log n, \log^2 n, \ldots$

$$2^{2^n}, \quad n!, \quad n2^n, \quad (3/2)^n, \quad (\log n)^{\log n} = \Theta(n^{\log\log n}),$$

$$n^3, \quad \log(n!) = \Theta(n\log n), \quad n = 2^{\log n},$$

$$\log^2 n, \quad \log n, \quad \sqrt{\log n}, \quad \log\log n,$$

$$n^{1/\log n} = \Theta(1)$$

例题

例1 设 $f(n)=\frac{1}{2}n^2-3n$， 证明 $f(n)=\Theta(n^2)$.

$$\lim_{n\to+\infty}\frac{f(n)}{n^2}=\lim_{n\to+\infty}\frac{\frac{1}{2}n^2-3n}{n^2}=\frac{1}{2}$$

根据定理1.1有 $f(n)=\Theta(n^2)$.

例题

例2　PrimalityTest(n)

输入：n, n为大于 2 的奇整数
输出：true 或者false

```
1.  s ← √n
2.  for j ←2 to s
3.      if j 整除 n
4.          then return false
5.  return true
```

假设计算 $\sqrt{n}$ 可以在$O(1)$时间完成，可以写$O(\sqrt{n})$，
不能写$\Theta(\sqrt{n})$

多项式时间的算法

- 多项式时间的算法

  时间复杂度函数为$O(p(n))$的算法，其中$p(n)$是n的多项式

- 不是多项式时间的算法

  不存在多项式 $p(n)$使得该算法的时间复杂度为 $O(p(n))$

  包含指数时间甚至更高阶的算法

多项式函数与指数函数

时间复杂度函数	问题规模					
	10	20	30	40	50	60
n	10^{-5}	$2*10^{-5}$	$3*10^{-5}$	$4*10^{-5}$	$5*10^{-5}$	$6*10^{-5}$
n^2	10^{-4}	$4*10^{-4}$	$9*10^{-4}$	$16*10^{-4}$	$25*10^{-4}$	$36*10^{-4}$
n^3	10^{-3}	$8*10^{-3}$	$27*10^{-3}$	$64*10^{-3}$	$125*10^{-3}$	$216*10^{-3}$
n^5	10^{-1}	3.2	24.3	1.7 分	5.2 分	13.0 分
2^n	.001 秒	1.0 秒	17.9 分	12.7 天	35.7 年	366 世纪
3^n	.059 秒	58 分	6.5 年	3855 世纪	$2*10^8$ 世纪	$1.3*10^{13}$ 世纪

表中默认单位为秒

多项式函数与指数函数

时间复杂度函数	1小时可解的问题实例的最大规模		
	计算机	快100倍的计算机	快1000倍的计算机
n	N_1	$100\ N_1$	$1000\ N_1$
n^2	N_2	$10\ N_2$	$31.6\ N_2$
n^3	N_3	$4.64\ N_3$	$10\ N_3$
n^5	N_4	$2.5\ N_4$	$3.98\ N_4$
2^n	N_5	$N_5 + 6.64$	$N_5 + 9.97$
3^n	N_6	$N_6 + 4.19$	$N_6 + 6.29$

10亿次/秒机器求解的问题

- 快速排序算法给10万个数据排序, 运算量约为 $10^5\times\log_2 10^5\approx 1.7\times 10^6$, 仅需$1.7\times 10^6/10^9=1.7\times 10^{-3}$秒.
- Dijkstra算法求解1万个顶点的图的单源最短路径问题, 运算量约为$(10^4)^2=10^8$, 约需$10^8/10^9=0.1$秒.
- 回溯法解100个顶点的图的最大团问题, 运算量为 $100\times 2^{100}\approx 1.8\times 10^{32}$, 需要$1.8\times 10^{32}/10^9=1.8\times 10^{21}$秒 $=5.7\times 10^{15}$年, 即5千7百万亿年!
- 1 分钟能解多大的问题. 1分钟60秒, 这台计算机可做用 快速排序算法可给2×10^9(即, 20亿)个数据排序, 用 Dijkstra 算法可解2.4×10^5个顶点的图的单源最短路径问题. 而用回溯法一天只能解41个顶点的图的最大团问题

问题的复杂度分析

多项式时间可解的问题与难解的问题

- 多项式时间可解的问题 ***P***

  存在着解***P*** 的多项式时间的算法

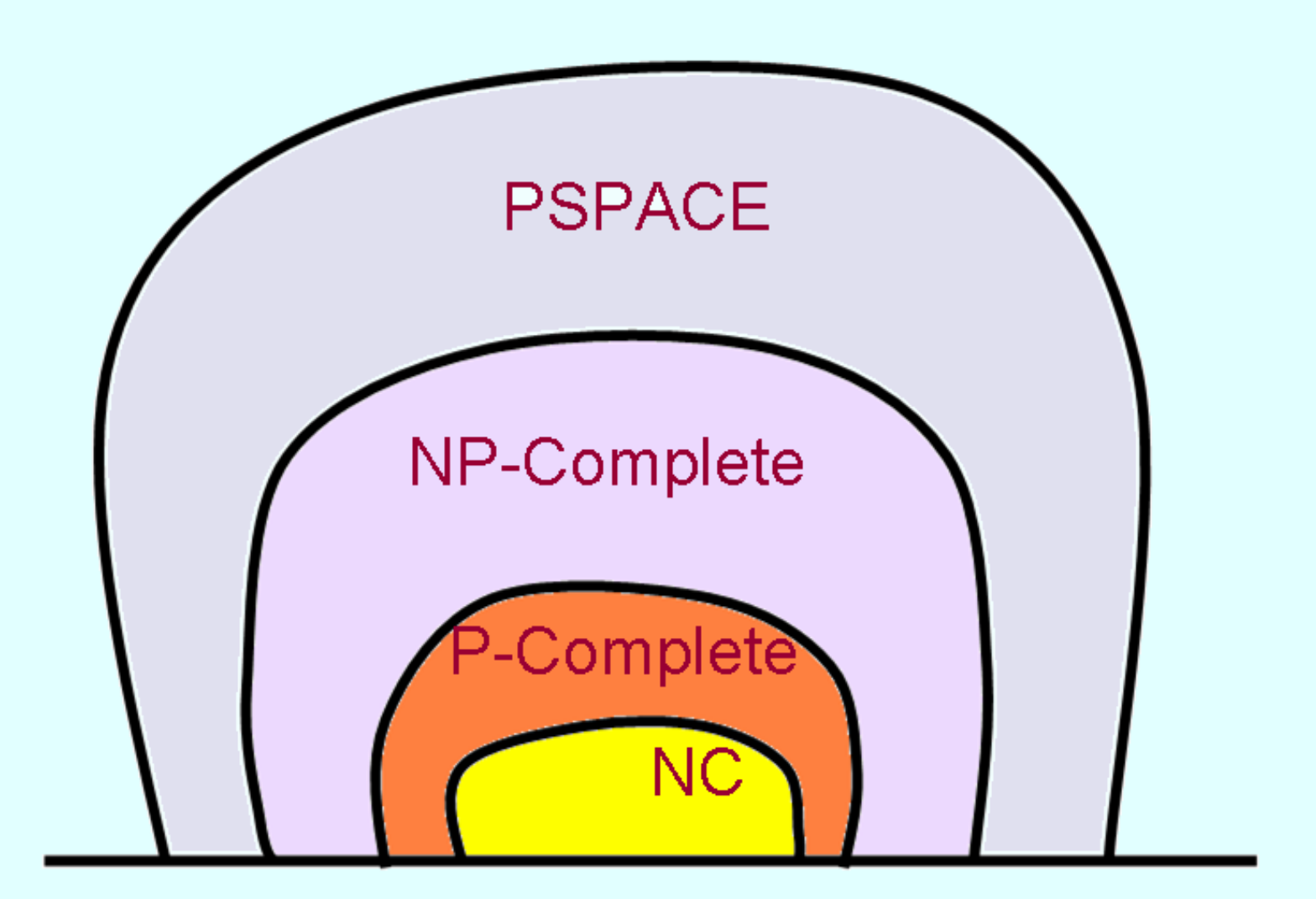
- 难解的问题***P***

  不存在解***P*** 的多项式时间的算法

- 实际上可计算的问题

  多项式时间可解的问题

不同复杂性类的基本层次结构



算法及计算复杂性理论的拓广

- 算法
  - 概率算法
  - 近似算法
  - 在线算法
  - 分布式算法
- 计算复杂性
  - 概率Turing机与概率复杂性
  - 近似求解的复杂性
  - 参数复杂性
  - 计数复杂性
  - 通信复杂性

数学基础：对数函数

符号：

$$\log n = \log_2 n, \quad (\lg n = \log_{10} n)$$

$$\log^k n = (\log n)^k$$

$$\log\log n = \log(\log n)$$

性质：

$$\log_b n = o(n^\alpha) \quad \alpha > 0$$

$$a^{\log_b n} = n^{\log_b a}$$

$$\log_k n = \Theta(\log_l n)$$

阶乘

Stirling公式 $$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1+\Theta\left(\frac{1}{n}\right)\right)$$

$$n! = o(n^n), \quad n! = \Omega(2^n)$$

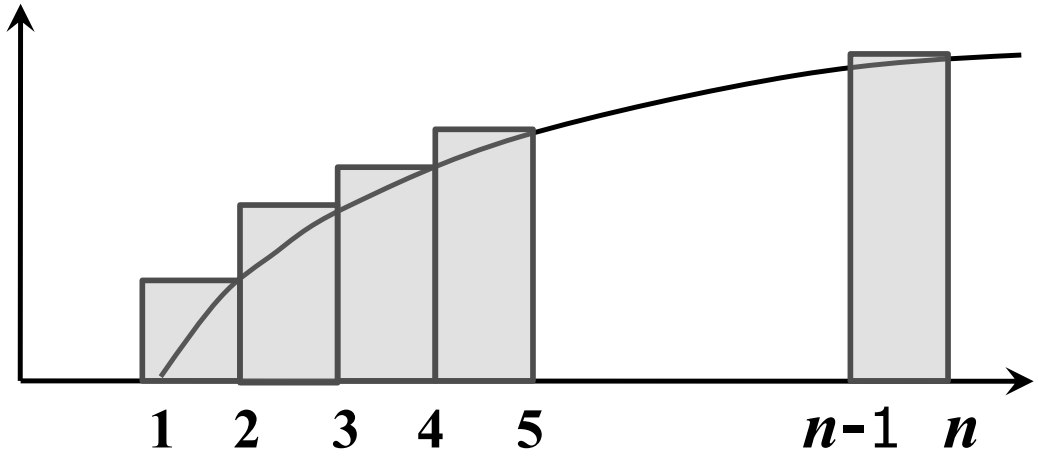
$$\log(n!) = \Theta(n\log n)$$

$$\log(n!) = \sum_{k=1}^{n}\log k$$

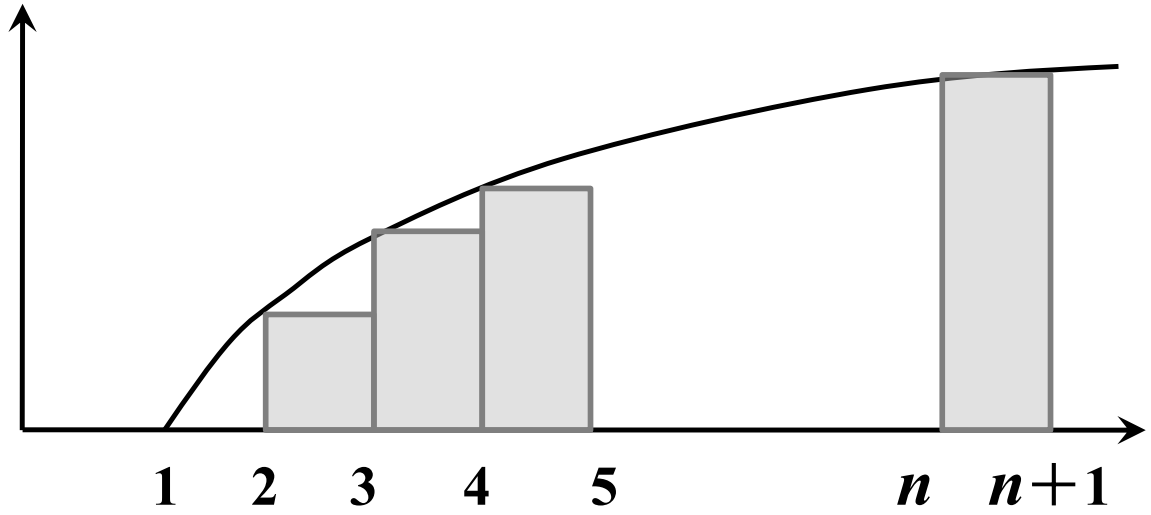
$$\geq \int_1^n \log x dx$$

$$= \log e(n\ln n - n + 1)$$

$$= \Omega(n\log n)$$



阶乘（续）



$$\log(n!) = \sum_{k=1}^{n} \log k \le \int_{2}^{n+1} \log x dx = O(n \log n)$$

取整函数

$\lfloor x \rfloor$：表示小于等于x的最大的整数

$\lceil x \rceil$：表示大于等于x的最小的整数

取整函数具有下述性质：

(1) $x-1<\lfloor x \rfloor \le x \le \lceil x \rceil < x+1$

(2) $\lfloor x+n \rfloor = \lfloor x \rfloor + n$, $\lceil x+n \rceil = \lceil x \rceil + n$，其中$n$为整数

(3) $$\left\lceil \frac{n}{2} \right\rceil + \left\lfloor \frac{n}{2} \right\rfloor = n$$

(4) $$\left\lceil \frac{\left\lceil \frac{n}{a} \right\rceil}{b} \right\rceil = \left\lceil \frac{n}{ab} \right\rceil, \quad \left\lfloor \frac{\left\lfloor \frac{n}{a} \right\rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor$$

求和公式

基本求和公式

$$\sum_{k=1}^{n} a_k = \frac{n(a_1 + a_n)}{2}, \quad \{a_k\}\text{为等差数列}$$

$$\sum_{k=0}^{n} aq^k = \frac{a(1-q^{n+1})}{1-q}, \quad \sum_{k=0}^{n} x^k = \frac{1-x^{n+1}}{1-x},$$

$$\sum_{k=0}^{\infty} aq^k = \frac{a}{1-q} \quad (q<1)$$

$$\sum_{k=1}^{n} \frac{1}{k} = \ln n + O(1)$$

估计和式上界的方法

放大法：

1. $\sum_{k=1}^{n} a_k \le n a_{\max}$

2. 假设存在常数 $r<1$，使得 对一切 $k \ge 0$成立 $\frac{a_{k+1}}{a_k} \le r$ ， 则

$$\sum_{k=0}^{n} a_k \le \sum_{k=0}^{\infty} a_0 r^k = a_0 \sum_{k=0}^{\infty} r^k = \frac{a_0}{1-r}$$

求和实例

例 求和

(1) $$\sum_{k=1}^{n-1}\frac{1}{k(k+1)}$$

解 (1) $$\sum_{k=1}^{n-1}\frac{1}{k(k+1)}=\sum_{k=1}^{n-1}\left(\frac{1}{k}-\frac{1}{k+1}\right)$$

$$=\sum_{k=1}^{n-1}\frac{1}{k}-\sum_{k=1}^{n-1}\frac{1}{k+1}=\sum_{k=1}^{n-1}\frac{1}{k}-\sum_{k=2}^{n}\frac{1}{k}=1-\frac{1}{n}$$

求和实例

(2) $\sum_{t=1}^{k} t2^{t-1}$

解 (2) $\sum_{t=1}^{k} t2^{t-1} = \sum_{t=1}^{k} t(2^t - 2^{t-1})$

$$= \sum_{t=1}^{k} t2^t - \sum_{t=1}^{k} t2^{t-1}$$

$$= \sum_{t=1}^{k} t2^t - \sum_{t=0}^{k-1} (t+1)2^t$$

$$= \sum_{t=1}^{k} t2^t - \sum_{t=0}^{k-1} t2^t - \sum_{t=0}^{k-1} 2^t$$

$$= k2^k - (2^k - 1)$$

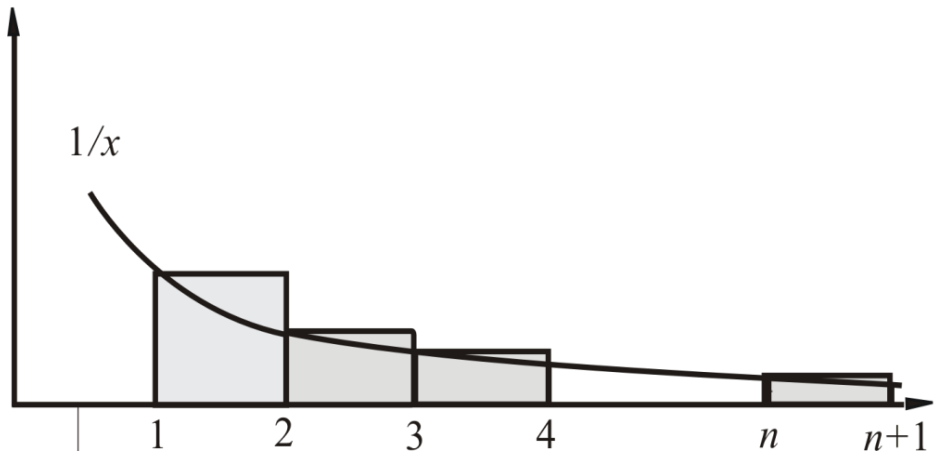
$$= (k-1)2^k + 1$$

估计和式渐近的界

估计 $\sum_{k=1}^{n}\frac{1}{k}$ 的渐近的界.

$$\sum_{k=1}^{n}\frac{1}{k}\geq\int_{1}^{n+1}\frac{dx}{x}$$

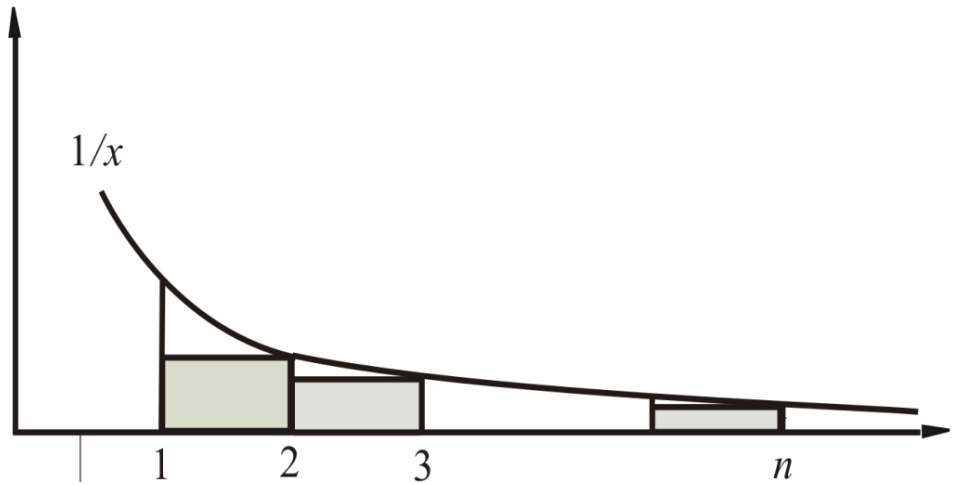
$$=\ln(n+1)$$



$$\sum_{k=1}^{n}\frac{1}{k}=\frac{1}{1}+\sum_{k=2}^{n}\frac{1}{k}$$

$$\leq 1+\int_{1}^{n}\frac{dx}{x}$$

$$=\ln n+1$$



总结

- 课程内容简介
- 理论和现实的可计算性
- 函数渐进的界
- 数学基础